LOOPGYM: ROLLOUT–COMBINE–FILTER TRAINING FOR TARGET-HIDDEN LOOP VERIFICATION

Anonymous authors
Paper under double-blind review

ABSTRACT

Valid loop invariants are closed under conjunction: useful clauses from different responses can prove a target even when no response does so alone. Yet standard training scores each response as an indivisible answer. We study this mismatch under *target-hidden loop verification*, where target assertions and postconditions Q are withheld during generation and training. LoopGym aligns both stages through rollout–combine–filter (RCF). At inference, RCF extracts and unions clauses, then uses syntax filtering and Houdini to retain an inductive subset before restoring Q. SFT distills a filtered multi-response union into one response. RL assigns credit within a rollout to its verifier-retained subset and across rollouts to complementary coverage beyond peers. Its target-independent strength signal comes from reachable executions and conservatively filtered synthetic negatives. On 832 C programs, five `gpt-5-nano` rollouts verify 74.76% of tasks, versus 61.78% for the strongest external baseline under each system's fixed native budget. In a paired Qwen3-8B evaluation, RL raises combine@10 from 52.76% to 56.97% despite lower pass@1 and reduces the rollouts needed to reach 95% of each policy's measured maximum performance from 30 to 10.

1 INTRODUCTION

Loop invariants turn unbounded executions into finite proofs, but a formally valid invariant can still be useless. An invariant is *valid* if it holds when execution first reaches the loop and remains true after every loop iteration. A verifier can check these obligations, yet its pass/fail outcome provides no graded measure of how much the invariant says: $true$ is valid for every loop but proves nothing about its behavior or desired outcome. A useful invariant must also be strong enough to establish downstream properties. Since the exact reachable-state set may be expensive to compute or inexpressible in the annotation language, generation must navigate many valid but weak over-approximations and receive credit for approaching stronger ones.

This search need not succeed in a single attempt. Validity is closed under conjunction: the conjunction of two valid invariants is valid and no weaker than either one. Useful clauses can therefore be accumulated across samples instead of demanding a complete answer from one response. Model responses expose such partial results as clauses: one may discover a range bound, another a relational equality, and together they may prove a target that neither proves alone. Clause-wise verification can discard invalid clauses without losing the rest. The natural unit of inference is therefore the filtered clause pool, not a complete response.

A complete response is also the wrong unit of training credit. A whole-response reward withholds credit from useful clauses whenever another clause in the same response fails. A reward computed independently for each response cannot distinguish a constraint that expands group coverage from one already repeated by every peer. Under repeated sampling, this can encourage the policy to reproduce easy high-reward clauses instead of exploring a clause pool whose members work together. Our central claim is that loop-invariant learning should optimize the verified composition obtained from a fixed rollout group, rather than treating every rollout as an independent final answer.

We study this claim under *target-hidden loop verification*. The model sees the function preconditions, executable prefix, loop guard, and body, but the target set Q is withheld until the generated invariant set has been fixed. The final test still uses the original task: after Q is restored, the verifier must

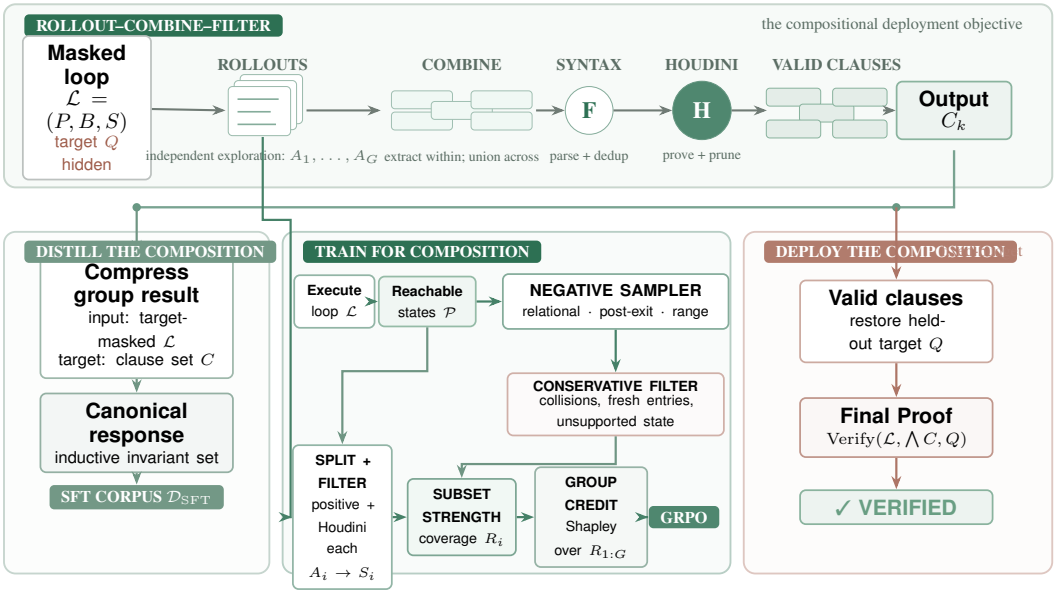

Figure 1: **Rollout–combine–filter is the organizing principle.** Target-hidden rollouts are parsed, unioned, and Houdini-filtered. SFT distills the combined set; RL rewards retained clauses within each rollout and complementary negative coverage across the group. Executions construct and clean only training-time reward signals; deployment restores Q only for the final proof.

establish the loop obligations and prove every target at its original program point. Hiding Q prevents a model from copying a target that happens to be valid or using target-specific proof failures to steer generation. It also rules out target success as a training reward, making a graded, target-independent measure of invariant strength necessary.

LoopGym organizes inference around *rollout–combine–filter* (RCF). Given a budget of k target-hidden responses, RCF extracts each invariant annotation as a clause, unions clauses across responses, removes syntax failures, and applies Houdini to retain a jointly valid subset. Only then is Q restored for the final proof. We measure this deployed object with combine@k: the fraction of programs verified after combining the first k responses. Unlike pass@k, which requires at least one response to contain the complete proof, combine@k preserves useful partial answers. The objective is to make this curve both higher and faster to converge, so a small rollout budget recovers the combinations otherwise found only through much broader sampling.

Training aligns credit with the two levels of this inference procedure. *Within* each rollout, we project its extracted clauses onto a verifier-retained subset; one rejected clause does not erase credit earned by the others. We score the strength of that subset using the execution-derived negative examples it rejects, without adding a reward merely for well-formed syntax or whole-response validity. *Across* rollouts, we apply a closed-form Shapley allocation to the union of their independently retained negative-example coverage: repeated rejections share credit, whereas a rejection supplied by fewer peers receives more. Thus the reward of one rollout depends on what the other rollouts in its Group Relative Policy Optimization (GRPO) group discover (Shao et al., 2024). Synthetic supervised fine-tuning (SFT) additionally distills a filtered multi-response union into one individual response. Together, these stages make sampled responses more useful to the clause pool that RCF combines at deployment.

Prior systems synthesize and prune candidate invariants at inference time, while learning-based methods train from verified outputs, solver feedback, or behavioral examples (Flanagan & Leino, 2001; Kamath et al., 2023; Cao et al., 2025; Si et al., 2020; Yan et al., 2025; Yang et al., 2026a; Huang et al., 2026). We instead train for the filtered clause pool used at inference: credit preserves useful clauses within a response and depends on complementary coverage across responses.

Our experiments deliberately separate standalone success from combined inference. On 832 integer C programs, five target-hidden `gpt-5-nano` responses with union and Houdini verify 622 programs

(74.76%), compared with 514 (61.78%) for the strongest external baseline under each system's fixed native budget. In the paired Qwen3-8B evaluation, reinforcement learning decreases pass@1 from 9.30% to 4.88% but increases combine@10 from 52.76% to 56.97%. It reduces the rollouts needed to reach 95% of each policy's measured maximum from 30 to 10. This reversal is evidence of the intended alignment: training improves the deployed composition rather than one standalone target-proving answer.

Contributions. This paper makes three contributions:

- We formulate target-hidden loop verification around a fixed-budget rollout–combine–filter objective. The deployed and evaluated object is the verified composition measured by combine@k, not an isolated response.
- We align learning credit with that object in both directions: the reward preserves useful clauses within imperfect rollouts and allocates complementary coverage across rollouts, without using Q.
- We show that this alignment improves combined verification and rollout efficiency, and audit the execution-derived examples that provide its graded strength signal.

2 PRELIMINARIES

Loop verification. A task consists of a loop $\mathcal{L} = (P, B, S)$ and a target set $Q = \{(\ell, q_\ell)\}$, where predicate q_ℓ occurs at assertion or postcondition location ℓ. Here P describes the loop-entry states induced by the function preconditions and executable prefix, B is the guard, and S is the body. Let $\mathrm{Reach}(\mathcal{L})$ be the reachable loop-head states, including the final guard test. An invariant I is *valid* (or inductive) when

$$P \Rightarrow I, \qquad \{I \wedge B\}\, S\, \{I\},$$

which ensures $\mathrm{Reach}(\mathcal{L}) \subseteq [\![I]\!]$ (Floyd, 1967; Hoare, 1969). The semantic ideal is an exact invariant $I^\star$ with $[\![I^\star]\!] = \mathrm{Reach}(\mathcal{L})$, although this set need not be finitely expressible in the annotation language. A valid invariant is Q-*sufficient* when annotating the loop with I lets the verifier prove every q_ℓ at ℓ. For one post-loop target this reduces to $I \wedge \neg B \Rightarrow q$. Thus validity is target-independent, whereas sufficiency is tested only after restoring Q.

Target-hidden verification. The generator and all intermediate scores use $\mathcal{L} = (P, B, S)$, not Q. Assertions, postconditions, executable assertion calls, and conventional error-label names are masked; non-contract comments, including source-provenance paths, are removed. Preconditions and executable loop semantics remain visible. The generated invariant is fixed before the untouched program is restored for final verification. We study scalar-integer C programs with one loop; Appendix A gives the scope and a motivating nonlinear example.

3 RELATED WORK

Candidate generation, filtering, and prompt-time composition. Classical invariant synthesis uses abstract interpretation, affine relations, templates, interpolation, recurrences, syntax-guided search, or counterexamples; execution-driven systems instead mine likely properties from traces (Cousot & Cousot, 1977; Karr, 1976; Colón et al., 2003; McMillan, 2003; Kincaid et al., 2017; Alur et al., 2013; Garg et al., 2014; Ernst et al., 2007). Houdini iteratively removes invalid candidates to retain an inductive subset (Flanagan & Leino, 2001). LLM workflows reuse this pattern: LOOPY couples proposals with Houdini and repair, IRANK ranks candidates, and CLAUSE2INV combines retained clauses while iteratively accumulating counterexamples (Kamath et al., 2023; Chakraborty et al., 2023; Cao et al., 2025). LAM4INV, ACINV, AUTOSPEC, SPECGEN, SESPEC, LEMUR, and BALDUR similarly organize generation with static analysis, symbolic execution, verification, or repair (Wu et al., 2024a; Liu et al., 2024b; Wen et al., 2024; Ma et al., 2025; Yang et al., 2026b; Wu et al., 2024b; First et al., 2023). These are inference workflows; LOOPGYM fixes all target-hidden rollouts before joint filtering, receives no verifier feedback between calls, and trains responses for fixed-group composition.

Specialized neural invariant learning. CODE2INV learns a construction policy from solver feedback; CLN2INV and G-CLN learn continuous representations of linear and nonlinear relations;

Algorithm 1 Rollout–combine–filter

Input: masked loop $\mathcal{L}$, parsed clause sequences $A_{1:G}$, cap K
Output: filtered clause set C

$C \leftarrow \bigcup_i \mathrm{Cap}_K(\mathrm{Normalize}(A_i))$
$C \leftarrow \mathrm{ConservativeDedup}(C)$
repeat
  $E \leftarrow \mathrm{ParserErrors}(\mathcal{L}, C)$; $C \leftarrow C \setminus E$
until $E = \varnothing$ or $C = \varnothing$
repeat
  $D \leftarrow \{c \in C : \neg\, \mathrm{WP}_{\mathrm{est,pres}}(\mathcal{L}, C, c)\}$; $C \leftarrow C \setminus D$
until $D = \varnothing$ or $C = \varnothing$
return C

and LIPUS combines RL-based pruning with SMT solving (Si et al., 2018; 2020; Ryan et al., 2020; Yao et al., 2020; Yu et al., 2023). These task-specific models optimize one candidate or construction trajectory. We instead post-train a general language model so independently sampled responses contribute jointly under a fixed budget.

Behavioral-example objectives for specification strength. COINS assesses specifications using trusted positive and mutation-derived negative input–output cases (Yang et al., 2026a); SPECRL rewards complete method-level specifications that reject impossible input–output pairs (Huang et al., 2026); it neither targets loop invariants nor derives negatives from loop dynamics. Both show that behavioral examples distinguish weak but valid specifications from stronger ones. LOOPGYM supplies that loop-specific construction, filters each imperfect response to an inductive clause subset, and conditions its credit on negative states already rejected by peer rollouts.

Language-model training from formal feedback. INVBENCH and Wonda use verified data for invariant fine-tuning; CODERL uses execution signals; and RE:FORM applies verifier-guided RL to individual Dafny proofs (Wei et al., 2025; Pinto et al., 2026; Le et al., 2022; Yan et al., 2025). Unlike whole-output verifier rewards, LOOPGYM trains toward a filtered clause pool by salvaging the inductive part of an imperfect response and rewarding its strength and complementary group coverage. This distinction matters because verifiable-reward RL can improve small-budget accuracy without expanding broad-sampling coverage (Yue et al., 2025).

4 METHOD

4.1 ROLLOUT–COMBINE–FILTER

For a target-masked loop $\mathcal{L}$, the model emits ACSL loop-invariant annotations (Baudin et al., 2021). Parsing extracts each `loop invariant ...;` annotation as one clause; normalization turns response i into $A_i = (a_{i1}, \ldots, a_{im_i})$. We cap it at K clauses, $\bar{A}_i = (a_{i1}, \ldots, a_{i,\min(m_i,K)})$, with overflow $o(A_i) = \max(0, m_i - K)$. Extraction makes the annotation, rather than the complete response, the unit that can survive filtering and earn credit.

Given G responses, RCF unions their clauses,

$$X_0 = \bigcup_{i=1}^{G} \bar{A}_i,$$

then iteratively removes syntax failures and applies Houdini to delete clauses whose establishment or preservation obligations fail (Flanagan & Leino, 2001). We write the resulting inductive subset as $\mathsf{Filter}_{\mathcal{L}}(X_0)$. Conjoining valid clauses preserves validity and can combine a bound from one response with a relation from another; filtering salvages such clauses from otherwise imperfect responses but cannot invent a missing relation. Algorithm 1 is shared by SFT synthesis and deployment.

RL scoring additionally removes clauses contradicted by observed reachable states $\mathcal{P}$ before Houdini:

$$\mathsf{PF}_{\mathcal{P}}(X) = \{a \in X : \forall p \in \mathcal{P},\ p \models a\}.$$

This positive filter is a cheap refutation step, not a validity proof; Frama-C/WP remains the authority. It is used only for RL scoring; deployment performs the same identifier-scope checks but no execution-based rejection. Appendix B gives normalization and filtering details.

4.2 Distilling Multi-Response Search

For each masked loop $\mathcal{L}$ in the training set $\mathcal{T}$, let $X_0(\mathcal{L})$ be a sampled response-group union and $C_{\mathcal{L}} = \mathsf{Filter}_{\mathcal{L}}(X_0(\mathcal{L}))$. We serialize every nonempty result as one SFT target:

$$\mathcal{D}_{\mathrm{SFT}} = \{(\mathcal{L}, \mathrm{serialize}(C_{\mathcal{L}})) : \mathcal{L} \in \mathcal{T},\ C_{\mathcal{L}} \neq \varnothing\}.$$

Neither input nor target contains Q: SFT serializes a Houdini-retained multi-response search result as one response. Execution states are used only for RL rewards, never as SFT answers.

4.3 Target-Independent Strength Examples

To grade inductive sets without Q, we execute a deterministic, precondition-checked input sweep and record reachable loop-head states $\mathcal{P}$. From these executions we construct three negative-example families: local one- or two-variable perturbations that break relations, continuations of the real body after a witnessed loop exit, and perturbations outside observed variable ranges. Conservative cleaning removes observed reachable states, possible fresh entries, duplicates, and candidates involving unsupported state. These examples are training signals, not proofs of global unreachability; unsupported or incomplete executions disable the affected family. Figure 3, Algorithm 2, and exact budgets appear in Appendix B.1.

Let $\mathcal{N}$ be the retained negative traces. For a retained clause set X, $\mathrm{rej}(X) \subseteq \mathcal{N}$ contains each trace on which at least one clause in X is false at some recorded state; singleton traces recover ordinary state rejection.

4.4 Clause-Decomposed, Group-Compositional Reward

For rollout i, define its verifier-retained subset and rejected negatives as

$$S_i = \mathsf{Filter}_{\mathcal{L}}(\mathsf{PF}_{\mathcal{P}}(\bar{A}_i)), \qquad R_i = \mathrm{rej}(S_i).$$

This projection implements *intra-rollout decomposition*: malformed or non-inductive clauses are removed without erasing useful clauses in the same response. When retained negatives exist, the reward adds no bonus merely for valid syntax or whole-response validity. The subset receives the dense, target-independent strength score

$$\mathrm{coverage}(A_i) = \frac{|R_i|}{|\mathcal{N}|}.$$

This and the following allocation apply when $\mathcal{N} \neq \varnothing$; when the sampler retains none, we use the binary fallback in Appendix B.2.

Independent coverage, however, rewards a complementary discovery exactly like one repeated by every peer. We therefore allocate the union of the standalone-retained rejection sets with the coverage game $v(T) = |\bigcup_{j \in T} R_j|/|\mathcal{N}|$. Let $f(n) = \sum_{j=1}^{G} \mathbf{1}[n \in R_j]$. The Shapley value of rollout i has the closed form (Shapley, 1953)

$$\phi_i = \frac{1}{|\mathcal{N}|} \sum_{n \in R_i} \frac{1}{f(n)}, \qquad \sum_{i=1}^{G} \phi_i = \frac{|\bigcup_i R_i|}{|\mathcal{N}|}. \tag{1}$$

Thus a unique rejection receives full credit, whereas $f(n)$ redundant rejections split it. This closed form uses already-computed rejection sets and adds no verifier calls. It implements *inter-rollout composition*: the same response earns less group credit when its verified constraints are already supplied by peers.

With conservative within-response duplicate count $d_{\mathrm{dup}}(A_i) = |\bar{A}_i| - |\,\mathrm{dedup}(\bar{A}_i)|$, the reward is

$$r_i = w_c\,\mathrm{coverage}(A_i) + w_g \phi_i - w_d d_{\mathrm{dup}}(A_i) - w_o o(A_i), \tag{2}$$

where $(w_c, w_g, w_d, w_o) = (1.0, 0.3, 0.02, 0.05)$. GRPO normalizes these group-conditioned rewards within the sampled response group. Shapley credit distinguishes equal-coverage rollouts when their rejection sets overlap differently, favoring less redundant coverage. It remains a target-independent behavioral surrogate rather than credit for final Q-sufficiency. Appendix B.2 gives the standard policy objective, duplicate normalization, and fallback behavior.

4.5 Deployment Objective

At deployment, RCF samples k masked responses and produces

$$X_k = \bigcup_{i=1}^{k} \bar{A}_i, \qquad C_k = \mathsf{Filter}_{\mathcal{L}}(X_k), \qquad I_k = \bigwedge_{a \in C_k} a.$$

Only then is Q restored. The target utility and inference objective are

$$U_Q(A_{1:k}) = \mathbf{1}[\mathrm{Verify}(\mathcal{L}, I_k, Q)], \qquad \max_{\pi} \mathbb{E}_{A_{1:k} \sim \pi(\cdot|\mathcal{L})}[U_Q(A_{1:k})].$$

Here Verify requires establishment, preservation, and every restored target at its original location; for one post-loop target, its last obligation is $I_k \wedge \neg B \Rightarrow q$. Its empirical counterpart is combine@k, which can succeed even when no standalone response is sufficient. Training therefore aims to raise this curve and reach its large-pool performance with fewer rollouts.

5 Experiments

We ask how combination scales with sampling (RQ1), transfers across models (RQ2), compares with specialized tools (RQ3), changes after RL (RQ4), and depends on reward and sampler design (RQ5).

5.1 Experimental Setup

Data and judge. The cleaned training artifacts contain 37,481 target-masked scalar-integer single-loop RL records; 3,200 form SFT targets. Evaluation contains 832 C programs: 316 linear and 50 nonlinear-arithmetic (NLA) programs from the Clause2Inv release, plus 466 integer programs from Loopy (Yu et al., 2023; Cao et al., 2025; Kamath et al., 2023). The overlap and prompt-sanitation audits are in Appendix E. We remove non-contract comments and mask all target predicates before generation, then restore the untouched source only after invariants are fixed. Success requires Frama-C/WP (Kirchner et al., 2015), under a 5-second per-obligation prover budget, to prove establishment, preservation, and every restored target; all failures count as zero.

Inference protocol. All LoopGym evaluation runs share target masking, the canonical prompt, clause processing, and Houdini; external tools retain their own target-hidden orchestration. Within LoopGym, API models retain service-default reasoning; all locally served checkpoints use non-thinking decoding to match data synthesis and training. Decoding uses temperature 1.0 and top-p 1.0. API calls cap completions at 16,384 tokens, including provider-internal reasoning tokens; local vLLM calls cap each response at 2,048 emitted tokens. We use no re-roll or target feedback. RQ1 reuses one saved batch of 100 stochastic responses per task. Appendix F gives benchmark provenance, serving, and training details.

Metrics. Throughout evaluation, success means Q-sufficiency under the final verifier. Pass@k is the probability that at least one of k responses proves Q. Whenever a saved pool contains c successful responses among $n = 100$, we use the unbiased estimator $\widehat{\text{pass@}}\,k = 1 - \binom{n-c}{k} / \binom{n}{k}$ for that program and average it over programs. Only responses that prove Q count as successes. Combine@k is the fraction of programs proved by unioning the first k responses and applying Houdini. Pass@k requires one response to contain the whole proof, whereas combine@k can combine clauses from different responses. We do not compute the exact all-subset analogue for combine@k, because it would require filtering $\binom{100}{k}$ unions per program; we evaluate the fixed saved prefix instead. For the measured budget grid $\mathcal{K} = \{1, 10, 30, 50, 100\}$, we report $k_{95} = \min\{k \in \mathcal{K} : C_\pi(k) \geq 0.95\, C_\pi(100)\}$. A smaller k_{95} means the retained pool reaches its large-budget performance sooner.

5.2 RQ1: How Do pass@k and combine@k Scale?

We sample 100 target-hidden responses per task from five base checkpoints: Qwen3-4B, Qwen3-8B, Qwen3-14B, Qwen3-30B-A3B (Yang et al., 2025), and Llama. Complete values and k_{95} appear in Appendix 9.

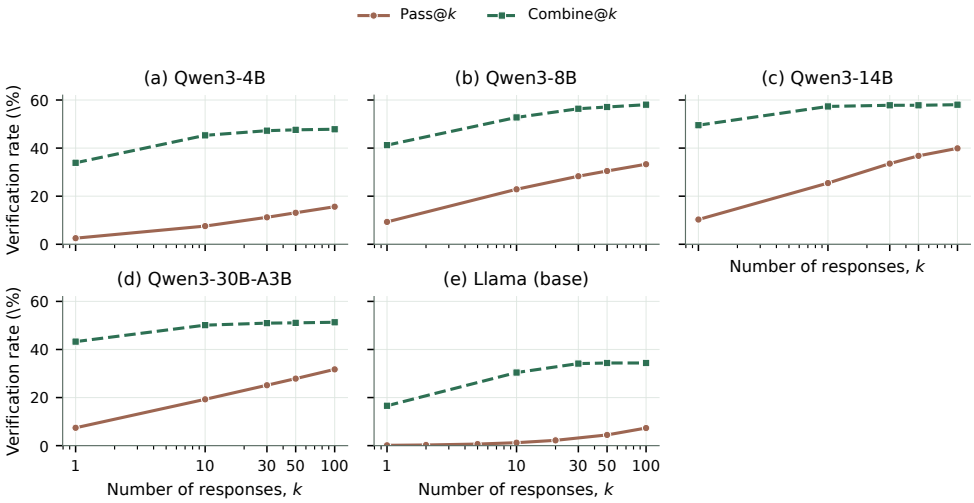

Figure 2: Pass@k (unbiased estimate) and prefix combine@k. Combination saturates by $k = 30$, while pass@k keeps growing.

At $k = 1$, prefix combine is 16.45–39.23 points above the unbiased pass estimate across checkpoints. Because these are different estimands rather than paired first-response outcomes, we treat this gap as descriptive; RQ2 reports the paired effect of processing the same response. In the retained response pools, every model reaches 95% of combine@100 by $k = 30$, with at most 1.68 additional points through $k = 100$. In contrast, pass@30 reaches only 71.8–85.0% of pass@100 on the four Qwen checkpoints and gains another 4.41–6.60 points by $k = 100$. Llama is harder in absolute terms—pass@100 is 7.33%—but shows the same compositional pattern: combine rises from 16.59% at $k = 1$ to 34.13% at $k = 30$, then changes by only 0.25 points. Combination thus front-loads useful coverage, whereas standalone proofs require wider sampling. The combine@100 ceilings (34.38–58.05%) further suggest that these pools remain bottlenecked by clause quality or diversity: extending the budget to 100 responses does not remove the gap.

5.3 RQ2: How Strong Is combine@1 Across Models?

The main neural experiment evaluates exactly one response per program. Pass@1 treats that response as one all-or-nothing invariant set. Combine@1 extracts invariant annotations as clauses, removes duplicates, and runs Houdini on the resulting pool. No pass@$k > 1$ or combine@$k > 1$ result is included in this RQ, so model quality is not confounded with a larger inference budget.

Model	pass@1	combine@1	Gain / regress.	Time / task	Avg. tokens
GPT-5-nano	452 (54.33%)	504 (60.58%)	123 / 71	66.60 s	6,928.30
GPT-5-mini	462 (55.53%)	604 (72.60%)	157 / 15	58.36 s	4,509.32
GPT-5	628 (75.48%)	636 (76.44%)	23 / 15	82.88 s	5,289.00
Claude Sonnet-4.6	467 (56.13%)	528 (63.46%)	TBD	59.88 s	1,193.55
DeepSeek-V4-Flash	428 (51.44%)	454 (54.57%)	32 / 6	166.98 s	11,084.44
Qwen3-14B	TBD	TBD	TBD	TBD	TBD
Qwen3-8B	TBD	TBD	TBD	TBD	TBD
Qwen3-4B	TBD	TBD	TBD	TBD	TBD
Llama (checkpoint TBD)	TBD	TBD	TBD	TBD	TBD

Table 1: Single-response cross-model results. Gain/regression counts pair the same response before and after clause processing. Time covers the end-to-end combine@1 pipeline; tokens cover the shared response.

Because pass@1 and combine@1 reuse the same response, their paired change isolates the net effect of clause extraction, normalization, and Houdini without additional model calls. The regression counts show that the realized pipeline is not monotone under finite verifier budgets, preventing a positive net change from being mistaken for uniform per-task improvement. API rows use provider-reported tokens and local rows use serving-tokenizer counts. Claude's time and token means come from a fixed seed-0 stratified sample of 20 programs (8 linear, 1 nonlinear, and 11 Loopy); its accuracy still uses all 832 programs. A separate non-reasoning GPT-5 diagnostic appears in Appendix 13.

5.4 RQ3: How Does LoopGym Compare with Specialized Tools?

We compare against the target-hidden pipelines of AutoSpec, SESpec, and Clause2Inv, and against Daikon 5.8.24 (Wen et al., 2024; Yang et al., 2026b; Cao et al., 2025; Ernst et al., 2007). All LLM-based rows use `gpt-5-nano`. Naive makes one call without filtering or refinement. Clause2Inv's unsupported inputs remain failures in the common 832-program denominator. Budgets are fixed before comparison but not matched: each system retains its predeclared native orchestration.

System	Linear / 316	NLA / 50	Loopy / 466	All / 832	Tokens / task	Time / task
AutoSpec	208 (65.82%)	4 (8.00%)	302 (64.81%)	514 (61.78%)	25,154.54	54.77 s
SESpec	180 (56.96%)	3 (6.00%)	231 (49.57%)	414 (49.76%)	44,807.05	117.31 s
Clause2Inv	63 (19.94%)	1 (2.00%)	0 (0.00%)	64 (7.69%)	385.80	29.04 s
Daikon	73 (23.10%)	0 (0.00%)	101 (21.67%)	174 (20.91%)	0	4.89 s
Naive	154 (48.73%)	4 (8.00%)	221 (47.42%)	379 (45.55%)	4,493.37	28.87 s
LoopGym (pass@1)	193 (61.08%)	7 (14.00%)	252 (54.08%)	452 (54.33%)	6,928.30	53.52 s
LoopGym (combine@1)	203 (64.24%)	7 (14.00%)	294 (63.09%)	504 (60.58%)	6,928.30	66.60 s
LoopGym (combine@5)	250 (79.11%)	7 (14.00%)	365 (78.33%)	622 (74.76%)	33,404.56	284.81 s
LoopGym (combine@10)	255 (80.70%)	7 (14.00%)	376 (80.69%)	638 (76.68%)	68,953.35	536.81 s

Table 2: Common target-hidden comparison using the same untouched inputs and Frama-C/WP final judge. Tokens are mean totals per task.

AutoSpec is the strongest external row at 61.78%. LoopGym combine@1 comes within 1.20 points while using 72.5% fewer tokens; combine@5 reaches 74.76% (+12.98 points) with 32.8% more tokens and substantially higher latency. The fixed rollout budget therefore exposes an explicit accuracy–cost trade-off. With the same `gpt-5-nano` backbone, combine@1 also improves the one-call Naive baseline from 45.55% to 60.58% without a repair conversation or growing context.

5.5 RQ4: How Much Does Reinforcement Learning Improve Inference?

We evaluate matched Qwen3-8B before–after RL paths from base and SFT initializations. Inference is fixed within each pair; RQ1 is not used to estimate the RL effect.

Initialization	Training stage	pass@1	combine@1	combine@10	combine@100	k_{95}
Base 8B	before RL (Zero)	9.30%	41.23%	52.76%	58.05%	30
Base 8B	after RL (RL-Zero)	4.88%	43.15%	56.97%	58.77%	10
SFT 8B	before RL (SFT)	TBD	TBD	TBD	TBD	TBD
SFT 8B	after RL (SFT+RL)	TBD	TBD	TBD	TBD	TBD

Table 3: Matched Qwen3-8B evaluation before and after RL.

RL-Zero trades lower pass@1 (9.30% to 4.88%) for higher combine@10 (52.76% to 56.97%). The within-policy k_{95} falls from 30 to 10, matching the deployed composition objective.

5.6 RQ5: What Drives the Training Gain?

Both ablations use Qwen3-8B and three matched seeds with all other training and inference settings fixed; Appendix G.3 reports per-seed combine@10.

Reward-function ablation. Variants progress from binary inductiveness to whole-response coverage, then add clause salvage, Shapley credit, and duplicate regularization. All use a 20-clause cap; coverage variants share the overflow cost, while the binary control remains $\{0, 1\}$.

Variant	pass@1	combine@1	combine@10	combine@100	k_{95}
Binary Inductiveness	TBD	TBD	TBD	TBD	TBD
Whole-Rollout Strength	TBD	TBD	TBD	TBD	TBD
Clause-Decomposed Strength	TBD	TBD	TBD	TBD	TBD
Compositional Credit	TBD	TBD	TBD	TBD	TBD
Regularized Compositional Credit (ours)	TBD	TBD	TBD	TBD	TBD

Table 4: Reward ablation on Qwen3-8B (three-seed means).

Negative-sampling ablation. *Random* uses uniform local perturbations; *Structured* uses relational, post-exit, and range families. Both share executions, filters, seed, and a 512-trace maximum; cleaning may leave different realized sizes.

Negative sampler	Reward var.	Collision	pass@1	combine@1	combine@10	combine@100	k_{95}
Random	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Structured (ours)	TBD	TBD	TBD	TBD	TBD	TBD	TBD

Table 5: Negative-sampler ablation under a shared maximum budget.

6 Conclusion and Limitations

LoopGym makes filtered clause composition the inference target: RCF combines rollouts, SFT distills their union, and target-independent RL rewards useful clauses and complementary coverage.

Evidence is limited to scalar-integer, single-loop C programs, Frama-C/WP and ACSL, and vLLM-served local checkpoints. Credit also filters rollouts separately whereas deployment filters their union. Appendices H and J detail these scope and reward approximations.

AI Use Statement

Generative AI tools are integral to the evaluated system: they produce the invariant candidates and synthetic SFT data described in the paper. They were also used during research to refine hypotheses, methodology, and experiments; formulate and check mathematical claims; implement

and test software; analyze and interpret experimental artifacts; prepare figures and tables; search and summarize literature; and draft, edit, and translate manuscript text. The authors reviewed all AI-assisted work and tested generated code. They take responsibility for independently validating the final content, empirical claims, and released artifacts.

Reproducibility Statement

The method and objective are specified in Section 4; Appendices B–F give the implementation defaults, prompt, training interface, data-overlap audit, and evaluation protocol. Appendix G organizes the available measurements and explicitly marks entries that still await final artifacts, including per-seed results.

References

Rajeev Alur, Rastislav Bodík, Garvit Juniwal, Milo M. K. Martin, Mukund Raghothaman, Sanjit A. Seshia, Rishabh Singh, Armando Solar-Lezama, Emina Torlak, and Abhishek Udupa. Syntax-guided synthesis. In *Proc. Formal Methods in Computer-Aided Design (FMCAD)*, pp. 1–8, 2013.

Patrick Baudin, Pascal Cuoq, Jean-Christophe Filliâtre, Claude Marché, Benjamin Monate, Yannick Moy, and Virgile Prevosto. ACSL: ANSI/ISO C specification language. `https://frama-c.com/html/acsl.html`, 2021.

Weining Cao, Guangyuan Wu, Tangzhi Xu, Yuan Yao, Hengfeng Wei, Taolue Chen, and Xiaoxing Ma. Clause2Inv: A generate-combine-check framework for loop invariant inference. *Proceedings of the ACM on Software Engineering*, 2(ISSTA):1009–1030, 2025. doi: 10.1145/3728920.

Saikat Chakraborty, Shuvendu K. Lahiri, Sarah Fakhoury, Madanlal Musuvathi, Akash Lal, Aseem Rastogi, Aditya Senthilnathan, Rahul Sharma, and Nikhil Swamy. Ranking LLM-generated loop invariants for program verification. In *Findings of the Association for Computational Linguistics: EMNLP*, pp. 9164–9175, 2023. doi: 10.18653/v1/2023.findings-emnlp.614.

Michael A. Colón, Sriram Sankaranarayanan, and Henny B. Sipma. Linear invariant generation using non-linear constraint solving. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 420–432. Springer, 2003.

Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proc. 4th ACM SIGACT-SIGPLAN Symp. on Principles of Programming Languages (POPL)*, pp. 238–252, 1977.

Michael D. Ernst, Jeff H. Perkins, Philip J. Guo, Stephen McCamant, Carlos Pacheco, Matthew S. Tschantz, and Chen Xiao. The Daikon system for dynamic detection of likely invariants. *Science of Computer Programming*, 69:35–45, 2007.

Emily First, Markus N. Rabe, Talia Ringer, and Yuriy Brun. Baldur: Whole-proof generation and repair with large language models. In *Proc. 31st ACM Joint European Software Engineering Conf. and Symp. on the Foundations of Software Engineering (ESEC/FSE)*, pp. 1229–1241, 2023.

Cormac Flanagan and K. Rustan M. Leino. Houdini, an annotation assistant for ESC/Java. In *Proc. Int. Symp. of Formal Methods Europe (FME)*, pp. 500–517. Springer, 2001.

Robert W. Floyd. Assigning meanings to programs. In *Proc. Symp. in Applied Mathematics: Mathematical Aspects of Computer Science*, volume 19, pp. 19–32, 1967.

Pranav Garg, Christof Löding, P. Madhusudan, and Daniel Neider. ICE: A robust framework for learning invariants. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 69–87. Springer, 2014.

C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12 (10):576–580, 1969.

Zhechong Huang, Zhao Zhang, Zeyu Sun, Huifeng Sun, and Yingfei Xiong. Reinforcement learning with negative tests as completeness signal for formal specification synthesis. *arXiv preprint arXiv:2604.05820*, 2026.

Adharsh Kamath, Aditya Senthilnathan, Saikat Chakraborty, Pantazis Deligiannis, Shuvendu K. Lahiri, Akash Lal, Aseem Rastogi, Subhajit Roy, and Rahul Sharma. Finding inductive loop invariants using large language models. *arXiv preprint arXiv:2311.07948*, 2023.

Michael Karr. Affine relationships among variables of a program. *Acta Informatica*, 6(2):133–151, 1976.

Zachary Kincaid, Jason Breck, Ashkan Forouhi Boroujeni, and Thomas Reps. Compositional recurrence analysis revisited. In *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, pp. 248–262, 2017.

Florent Kirchner, Nikolai Kosmatov, Virgile Prevosto, Julien Signoles, and Boris Yakobowski. Frama-c: A software analysis perspective. *Formal Aspects of Computing*, 27(3):573–609, 2015.

Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven C. H. Hoi. CodeRL: Mastering code generation through pretrained models and deep reinforcement learning. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Chang Liu, Xiwei Wu, Yuan Feng, Qinxiang Cao, and Junchi Yan. Towards general loop invariant generation: A benchmark of programs with memory manipulation. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2024a.

Ruibang Liu, Minyu Chen, Ling-I Wu, Jingyu Ke, and Guoqiang Li. Enhancing automated loop invariant generation for complex programs with large language models. *arXiv preprint arXiv:2412.10483*, 2024b.

Lezhi Ma, Shangqing Liu, Yi Li, Xiaofei Xie, and Lei Bu. SpecGen: Automated generation of formal program specifications via large language models. In *Proc. Int. Conf. on Software Engineering (ICSE)*, 2025.

Kenneth L. McMillan. Interpolation and SAT-based model checking. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 1–13. Springer, 2003.

Ido Pinto, Yizhak Yisrael Elboher, Haoze Wu, Nina Narodytska, and Guy Katz. Not all invariants are equal: Curating training data to accelerate program verification with SLMs. *arXiv preprint arXiv:2603.15510*, 2026.

Gabriel Ryan, Justin Wong, Jianan Yao, Ronghui Gu, and Suman Jana. CLN2INV: Learning loop invariants with continuous logic networks. In *Proc. Int. Conf. on Learning Representations (ICLR)*, 2020.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Lloyd S. Shapley. A value for n-person games. In Harold W. Kuhn and Albert W. Tucker (eds.), *Contributions to the Theory of Games II*, volume 28 of *Annals of Mathematics Studies*, pp. 307–317. Princeton University Press, 1953.

Xujie Si, Hanjun Dai, Mukund Raghothaman, Mayur Naik, and Le Song. Learning loop invariants for program verification. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, pp. 7751–7762, 2018.

Xujie Si, Aaditya Naik, Hanjun Dai, Mayur Naik, and Le Song. Code2Inv: A deep learning framework for program verification. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 151–164. Springer, 2020.

Anjiang Wei, Tarun Suresh, Tianran Sun, Haoze Wu, Ke Wang, and Alex Aiken. InvBench: Can LLMs accelerate program verification with invariant synthesis? *arXiv preprint arXiv:2509.21629*, 2025.

Cheng Wen, Jialun Cao, Jie Su, Zhiwu Xu, Shengchao Qin, Mengda He, Haokun Li, Shing-Chi Cheung, and Cong Tian. Enchanting program specification synthesis by large language models using static analysis and program verification. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 302–328. Springer, 2024.

Guangyuan Wu, Weining Cao, Yuan Yao, Hengfeng Wei, Taolue Chen, and Xiaoxing Ma. LLM meets bounded model checking: Neuro-symbolic loop invariant inference. In *Proc. IEEE/ACM Int. Conf. on Automated Software Engineering (ASE)*, 2024a.

Haoze Wu, Clark Barrett, and Nina Narodytska. Lemur: Integrating large language models in automated program verification. In *Proc. Int. Conf. on Learning Representations (ICLR)*, 2024b.

Chuanhao Yan, Fengdi Che, Xuhan Huang, Xu Xu, Xin Li, Yizhi Li, Xingwei Qu, Jingzhe Shi, Chenghua Lin, Yaodong Yang, Binhang Yuan, Hang Zhao, Yu Qiao, Bowen Zhou, and Jie Fu. Re:Form—reducing human priors in scalable formal software verification with RL in LLMs: A preliminary study on Dafny. *arXiv preprint arXiv:2507.16331*, 2025.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Fanpeng Yang, Xing Li, Shuling Wang, Jie An, Zeyu Sun, Shenghua Feng, Wenhan Wang, Weiyi Wang, Naijun Zhan, and Fanjiang Xu. How powerful are LLMs in generating formal program specifications? In *Proc. Int. Conf. on Machine Learning (ICML)*, 2026a.

Fanpeng Yang, Xu Ma, Shuling Wang, Xiong Xu, Qinxiang Cao, Naijun Zhan, Xiaofeng Li, and Bin Gu. Integrating symbolic execution with LLMs for automated generation of program specifications. *arXiv preprint arXiv:2506.09550*, 2026b.

Jianan Yao, Gabriel Ryan, Justin Wong, Suman Jana, and Ronghui Gu. Learning nonlinear loop invariants with gated continuous logic networks. In *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, pp. 106–120, 2020.

Shiwen Yu, Ting Wang, and Ji Wang. Loop invariant inference through SMT solving enhanced reinforcement learning. In *Proc. ACM SIGSOFT Int. Symp. on Software Testing and Analysis (ISSTA)*, pp. 175–187, 2023. doi: 10.1145/3597926.3598047.

Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? In *Advances in Neural Information Processing Systems*, volume 38, 2025.

A Scope and Motivating Example

We focus on numerical loops because their arithmetic equalities and inequalities, including polynomial relations, admit scalable automatic checking. This controlled setting factors out arrays, heaps, recursive data structures, auxiliary functions, and inductive predicates, allowing the study to isolate algebraic discovery and loop dynamics. Concretely, we consider one braced `while` loop over scalar C integers, including nonlinear updates. Even this restricted setting can hide nonlinear relations across coupled recurrences. For example:

```c
void f(int k, int z) {
  int x = 1, y = 1, c = 1;
  while (c < k) { c++; x = x*z + 1; y = y*z; }
  /*@ assert 1+x*z-x-z*y == 0; */
}
```

When visible, the assertion can be copied verbatim as an invariant; when hidden, its nonlinear relation must be recovered from the coupled updates.

B Implementation Details

Table 6 lists the implemented defaults. Execution sampling is used only to compute reinforcement-learning rewards; it does not construct the synthetic SFT corpus. SFT synthesis, RL, and inference share the rollout–combine–filter implementation: the same annotation extraction, per-rollout response cap, normalization, candidate combination, iterative syntax filtering, conservative semantic deduplication, and Houdini pruning. SFT serializes the filtered combination, RL scores the responses that populate it, and inference restores Q only after filtering. Inference does not construct execution examples. The operators are shared, but RL applies Houdini once per response for credit assignment whereas SFT and inference apply it after union; Appendix J analyzes this ordering mismatch.

The exact near-input tier order is `0, 1, 2, -1, 3, 5, 8, -2, 10, 17, 4, -3, 25, 6, 40, 7, -5, 13, 50, 9`.

B.1 Negative-Trace Sampling

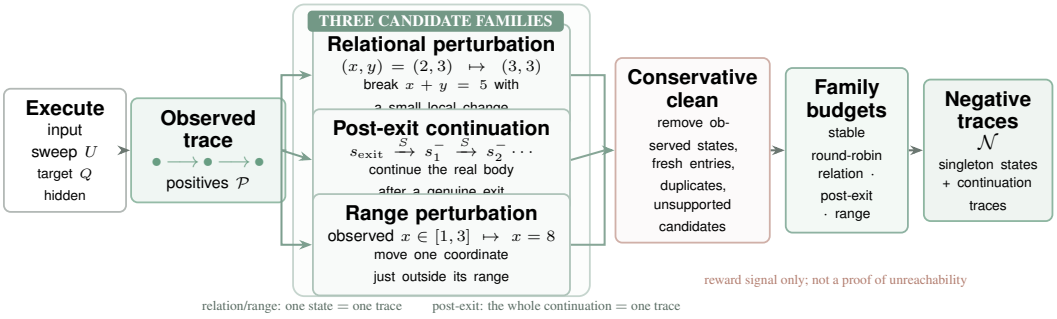

Figure 3: **Construction of target-hidden negative traces.** Concrete executions provide observed reachable states. Three transformations probe relational structure, behavior past a genuine exit, and sampled bounds. Conservative cleaning and per-family budgets produce $\mathcal{N}$, which is used only for RL reward computation and never exposes Q.

Algorithm 2 summarizes the sampler used for RL. Trace records real loop-head states and, after each genuine exit, continues the real loop body to obtain one post-exit trace. $\mathrm{Perturb}_\Theta$ applies the configured single-variable and pairwise changes, while $\mathrm{RangeEscape}$ moves one variable just outside its sampled range. The exact perturbations and budgets are those in Table 6.

Constant	Value	Role
runs requested	12	input executions; deterministic no-input programs collapse to one, and oracle-body sampling doubles the request
sampler seed	0	one canonical example set per reward request unless overridden
input tiers	20 fixed values	deterministic near-input schedule listed below; unconstrained tier candidates are clamped to $[-64, 64]$
far probes	3, plus 2 ordered multi-parameter probes	seed-hashed input-envelope coverage, magnitude ≤ 512
relation deltas	dense: $\pm\{1, 2\}$; terminal: $\pm\{1, 2, 3, 5, 8\}$	single-axis and pairwise perturbations; terminal bases require a witnessed final transition rather than a forward dense window
dense window	3	sampled successors required for a relation base
relation base cap	96	bases stratified across positives
range-perturbation steps	$\pm\{5, 8, 13, 21, 34\}$	candidates kept only outside the sampled range
post-exit steps	24	continuation length after a genuine exit (one negative trace per run)
negative-example budgets	320 (≤ 64 fallback) / 64 / 128	relational / post-exit / range examples; stable round-robin within each kind
iteration cap	2×10^6	divergence safety net (capped runs flagged)
(w_c, w_g)	$(1.0, 0.3)$	negative rejection / Shapley group-credit weights
w_d	0.02	cost per conservative duplicate; unique zero-coverage clauses are not penalized
K	20 clauses	response cap applied independently to every generated response
w_o	0.05	overflow cost per clause after K
Houdini iterations	≤ 500	greatest-inductive-subset pruning
WP configuration	Alt-Ergo, 30 s, Typed model	current implementation default and per-goal budget

Table 6: Defaults for execution sampling (§4.3), reward computation (§4.4), inference, and verification.

Algorithm 2 Target-hidden negative-trace sampling

Input: masked loop $\mathcal{L} = (P, B, S)$, input schedule U, configuration Θ
Output: reachable states $\mathcal{P}$, negative traces $\mathcal{N}$

$\mathcal{T}, \mathcal{C}_{\text{post}}, c \leftarrow \text{Trace}(\mathcal{L}, U, \Theta)$ ▷ *c: some run was capped*
$\mathcal{P} \leftarrow \text{UniqueHeads}(\mathcal{T})$; $M \leftarrow \text{SafeModifiedVars}(S)$
if $M = \varnothing$ or S has unsupported state **then return** $(\mathcal{P}, \varnothing)$
$\mathcal{S} \leftarrow \text{StratifiedSample}(\mathcal{P})$
$C_{\text{rel}} \leftarrow \text{Perturb}_\Theta(\text{Dense}(\mathcal{S}), M)$
if $\neg c$ **then** $C_{\text{rel}} \leftarrow C_{\text{rel}} \cup \text{Perturb}_\Theta(\text{Terminals}(\mathcal{T}), M)$
$C_{\text{range}} \leftarrow \text{RangeEscape}(\mathcal{S}, M, \mathcal{P}, \Theta)$ if $\neg c$, else $\varnothing$
$(C_{\text{rel}}, \mathcal{C}_{\text{post}}, C_{\text{range}}) \leftarrow \text{Clean}_{\mathcal{P}}(C_{\text{rel}}, \mathcal{C}_{\text{post}}, C_{\text{range}})$
$N_{\text{rel}}, \mathcal{N}_{\text{post}}, N_{\text{range}} \leftarrow \text{BudgetAndRoundRobin}(C_{\text{rel}}, \mathcal{C}_{\text{post}}, C_{\text{range}}, \Theta)$
$\mathcal{N} \leftarrow \{\langle s \rangle : s \in N_{\text{rel}}\} \cup \mathcal{N}_{\text{post}} \cup \{\langle s \rangle : s \in N_{\text{range}}\}$
return $(\mathcal{P}, \mathcal{N})$

$\text{Clean}_{\mathcal{P}}$ removes observed reachable states, possible fresh loop entries, duplicates, and unsupported candidates, using family priority relation, post-exit, then range. A cleaned post-exit continuation remains one trace; relation and range candidates become singleton traces. Within the relation budget, candidates that preserve the guard truth value and remain inside the sampled coordinate ranges are selected first.

B.2 Group-Relative Optimization Details

For sampled sequences $o_{1:G} \sim \pi_{\theta_{\text{old}}}(\cdot \mid \mathcal{L})$, we normalize Equation 2 within its group,

$$\hat{A}_i = \frac{r_i - \text{mean}_j\, r_j}{\text{std}_j\, r_j + 10^{-6}},$$

and use the clipped GRPO objective (Shao et al., 2024)

$$\mathcal{J}(\theta) = \mathbb{E}\left[\frac{1}{G}\sum_i \frac{1}{|o_i|}\sum_t \min\Big(\rho_{i,t}\hat{A}_i, \text{clip}(\rho_{i,t}, 1-\varepsilon, 1+\varepsilon)\hat{A}_i\Big) - \beta D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}})\right]. \quad (3)$$

Here $\rho_{i,t}$ is the token-level ratio to the rollout policy and π_{ref} is frozen. Duplicate detection normalizes comparison direction, equality sides, parentheses, immediate commutative operands, harmless identities, and Boolean association, but avoids transformations that require additional arithmetic assumptions. Unique zero-coverage clauses are not penalized because they may support another clause or the hidden target. Both coverage and Shapley credit lie in $[0, 1]$, so the unpenalized full reward lies in $[0, 1.3]$; duplicate and overflow costs can lower it. When the sampler retains no negatives, scoring falls back to binary inductiveness for a nonempty response. Combined-pool coverage is logged as a diagnostic but is not copied into every rollout reward.

C Generation Prompt

The canonical evaluation template appears below; every target and non-contract comment is stripped before insertion, and general annotation rules are supplied by `system_prompt.txt`. We rebuild both training sets with this single prompt pair. From archival pools of 37,958 RL and 3,454 SFT records, the final artifacts retain 37,481 and 3,200 records, respectively. Fourteen records in each archive fall outside the scalar-integer, single-loop interface. A Frama-C kernel pass over 36,301 unique RL sources removes another 463 rows with C/ACSL parse or name-resolution errors. SFT cleaning removes 240 rows whose answers contain no surviving clause.

SFT answers are parsed into clauses, conservatively deduplicated, stripped of malformed, out-of-scope, tautological, and dominated constant-bound clauses, then checked by the same Frama-C/WP Houdini filter used by the method. The 5-second limit applies to each proof obligation. This reduces 32,883 archived clauses to 25,781 verified, non-tautological clauses, with at most 20 clauses per answer. A final conservative theorem-prover pass removes 431 universally true clauses from the verified set; five answers then become empty. For archived helper calls, 65 fixed-exponent occurrences are expanded into explicit multiplication; 22 resulting clauses remain in 20 final examples. Symbolic exponents are never approximated by finite expansion. Instead, when two equations share the same symbolic power, we eliminate that term to derive a power-free polynomial relation and verify it on the corresponding program; 175 such relations remain in 104 examples. The remaining 2,083 symbolic-power clauses cannot be represented equivalently under the released scalar interface and are removed. The artifact records every per-program rewrite and verdict.

Contract-level audits find no target clause or non-contract comment in any final RL or SFT model prompt, and all rows contain the same current system and user templates. The archival full source remains only in RL reward metadata; the service strips its target before sampling and inductiveness filtering. Likewise, model-visible evaluation sources contain no provenance or outcome comments. These cleaned artifacts require retraining; results from checkpoints trained on the archived prompts are not evidence for the final protocol.

Generation prompt

```
You are given a C function. Produce the strongest inductive ACSL loop
invariants that capture the loop's behavior (conservation/relational laws
+ tight progress bounds). Output ONLY invariant lines, each formatted
exactly as:
loop invariant <expr>;
Output at most 20 invariant lines.

Program:
{program}
```

D TRAINING INTERFACE

The training service is packaged as a self-contained, CPU-only container with gcc, the verifier, Why3, and its prover backend. It exposes `POST /reward`, accepting program source, an array of model responses, `reward_variant` in {`binary`, `whole_coverage`, `base`, `base_shapley`, `full`}, and `sampler.negative_sampler` in {`random`, `structured`}. The two switches are independent; the reward defaults to `full` and the sampler to `structured`. The response exposes the selected modes and Shapley credit explicitly:

```
{program, n_positives, n_negatives, filter_mode,
 reward_variant, negative_sampler, reward_mode,
 rollout_rewards[], base[], shapley_credit[],
 redundant_clauses[], redundancy_penalty[],
 generated[], accepted[], overflow[], overflow_penalty[],
 rollouts[], batch_score}
```

Responses may be raw LLM text, invariant lists, or annotated code; parsing is uniform. An offline scorer provides the same generation reward over JSONL/Parquet rollout dumps.

E Training–Evaluation Overlap Audit

The 37,481 cleaned training records contain 35,838 unique source strings after whitespace normalization and include every program used for SFT. We compare this pool against all 832 evaluation files after removing Q, exactly as it is removed before model inference.

E.1 Layered Matching Protocol

A single exact-string check can miss renamed or lightly rewritten copies. We therefore apply four increasingly aggressive program-level normalizations, all preserving `requires` clauses.

1. **Target-hidden source** removes Q and non-contract comments, then normalizes whitespace.
2. **Near-exact tokens** additionally anonymize the loop function name and tokenize the remaining source.
3. **Alpha-normalized source** renames identifiers in first-use order and maps the benchmark nondeterministic-function names to one token, while preserving constants, operators, statement order, and nesting.
4. **Constant-abstracted source** retains 0 and 1 but maps every other integer literal to one token. This catches copies changed only by variable names and nontrivial numeric constants.

We separately measure control-flow similarity. The *coarse profile* records whether the loop guard is constant, numbers of `if`/`else` nodes and compound conditions capped at three, and the presence of `break`, `continue`, `return`, `goto`, and nested loops. The *ordered control-flow skeleton* retains the order and brace nesting of control nodes and replaces each contiguous block of ordinary statements by one token. Its stricter variant also retains comparison and logical operators in branch conditions. These control-flow signatures intentionally discard data flow and arithmetic and are similarity diagnostics, not program-identity tests. The accompanying audit artifact records SHA-256 hashes of both training files and the complete evaluation corpus.

E.2 Overlap Results

Representation	Shared signatures	Matched evaluation files
Target-hidden source	0	0 (0.0%)
Near-exact token skeleton	0	0 (0.0%)
Alpha-normalized source	0	0 (0.0%)
Constant-abstracted alpha source	0	0 (0.0%)
Coarse control-flow profile	21	762 (91.6%)
Ordered control-flow skeleton	21	524 (63.0%)
Skeleton + condition-operator shape	21	421 (50.6%)

Table 7: Layered training–evaluation overlap audit. Program-level representations preserve data flow; control-flow-only representations deliberately omit it.

No evaluation program matches the training pool at any program-level stage. In particular, overlap remains zero after all identifiers are alpha-renamed and after nontrivial constants are abstracted. This rules out exact copies, function-name or formatting variants, variable renamings, and copies changed only by numeric constants under the stated normalizations.

Control flow alone overlaps substantially. The coarse profiles of 762 of 832 evaluation files occur in training; preserving statement order and nesting reduces this to 524 files, and retaining condition-operator shapes reduces it to 421. This is expected for a scalar single-loop language: straight-line loops, one-branch loops, and `if--else` updates form a small reusable vocabulary. The contrast is the relevant observation: training covers common control-flow forms, but every match disappears once arithmetic and data-flow structure are retained. The observed control-flow overlap is therefore consistent with task coverage and does not by itself establish memorization.

Scope and residual risk. This audit covers the released fine-tuning data, not unknown base-model pretraining corpora. It may also miss algebraically equivalent updates, reordered independent statements, transformed loops, or distant semantic clones. We therefore report no overlap under the stated normalizations rather than claiming complete semantic independence.

F Experimental Protocol Details

The 37,481 RL records retain corpus-local identifiers but not per-record upstream-source identifiers. The separately retained SFT artifact contains 3,200 unique sources, of which 2,600 also occur in the RL artifact. We therefore characterize the training pool by content and audit its overlap, rather than inferring provenance from its IDs. The 832-file evaluation workload contains 316 linear programs (133 from Code2Inv, 84 from SyGuS 2019, and 99 from SV-COMP 2024), 50 nonlinear programs (30 from LIPuS and 20 from SV-COMP 2024), and the 466 integer programs in Loopy's original 469-program corpus. Three floating-point programs are excluded. Unsupported inputs remain failures in the common denominator; because the collections reuse earlier benchmarks, the files are not claimed to be 832 independently sourced problems.

Before model invocation, we remove `assert` and `ensures` predicates, executable assertion calls, conventional error-label names, and non-contract comments, including source-provenance paths; preconditions and executable semantics remain visible. LoopGym evaluation runs use the canonical generation template, extractor, semantic deduplication, and Houdini implementation; external tools retain their own orchestration. Each model retains its serving interface and chat template. In LoopGym, API calls do not override `reasoning_effort`, `thinking`, or equivalents; all locally served checkpoints disable thinking. We set temperature 1.0 and top-p 1.0, without an additional top-k, repetition penalty, re-roll, or target feedback. The cap is 16,384 completion tokens for API models, including any provider-internal reasoning tokens, and 2,048 emitted tokens for local vLLM models. Each RQ1 curve reuses the same stochastic 100-response batch per task; no per-response sampling seed is fixed.

Training configuration. Table 8 separates settings recoverable from the released pipeline from fields that still require the original trainer manifest. The final submission must replace every TBD; reward and sampler constants are already reported in Table 6.

Field	Value	Scope or required provenance
Backbone ID and revision	TBD	exact Qwen3-8B repository and immutable revision
SFT optimizer / schedule	TBD	optimizer, learning rate, warmup, decay, and precision
SFT batch / duration	TBD	global batch, accumulation, epochs or updates, and sequence length
RL algorithm	GRPO	group-normalized objective in Equation 3
RL optimizer / schedule	TBD	optimizer, learning rate, warmup, decay, and precision
Rollout group size G	TBD	responses sharing one group-relative update
RL batch / duration	TBD	prompt batch, accumulation, effective batch, and update count
Clipping / KL	TBD	ε, β, and reference-policy update convention
Training seeds	TBD	seed for each reported checkpoint and ablation run
Hardware / software	TBD	GPU type and count, trainer, vLLM, CUDA, and framework versions
Local rollout decoding	non-thinking; $T = 1$, top-$p = 1$; 2,048 tokens	data synthesis, RL, and local evaluation

Table 8: Training configuration and required run-manifest fields.

G Complete Experimental Data

This section collects the per-model, per-stratum, and per-seed measurements behind the main aggregates. Entries marked TBD await the final evaluation artifacts; no missing value is used in the analysis.

G.1 Probe Curves

Table 9: Complete probe results. k_{95} is the smallest measured $k \in \{1, 10, 30, 50, 75, 100\}$ at which combine@k reaches 95% of its value at $k = 100$.

Model	pass@1	pass@10	pass@30	pass@50	pass@100	combine@1	combine@10	combine@30	combine@50	combine@75	combine@100	k_{95}
Qwen3-4B	2.53%	7.56%	11.21%	13.10%	15.62%	33.89%	45.31%	47.24%	47.60%	47.72%	47.84%	30
Qwen3-8B	9.30%	22.87%	28.30%	30.47%	33.29%	41.23%	52.76%	56.37%	57.09%	57.57%	58.05%	30
Qwen3-14B	10.29%	25.45%	33.55%	36.78%	39.90%	49.52%	57.33%	57.81%	57.81%	57.93%	58.05%	10
Qwen3-30B-A3B	7.43%	19.27%	25.13%	27.88%	31.73%	43.27%	50.12%	50.96%	51.08%	51.20%	51.32%	10
Llama (checkpoint TBD)	0.14%	1.23%	–	4.46%	7.33%	16.59%	30.41%	34.13%	34.38%	34.38%	34.38%	30

The retained Llama artifact reports pass@2, pass@5, and pass@20 as 0.28%, 0.66%, and 2.21%, respectively; pass@30 was not included in the supplied summary. Its combine numerators at $k = 1, 10, 30, 50, 75, 100$ are 138, 253, 284, 286, 286, and 286 out of 832 tasks.

G.2 Reinforcement-Learning Results

Table 10: Complete comparison before and after RL. The Zero and SFT initializations are evaluated under the same decoding and verification configuration.

Path	Model	pass@1	combine@1	combine@10	combine@50	combine@100	k_{95}
Zero	Base 8B	9.30%	41.23%	52.76%	57.09%	58.05%	30
Zero	Base 8B + RL	4.88%	43.15%	56.97%	58.77%	58.77%	10
SFT	SFT 8B	TBD	TBD	TBD	TBD	TBD	TBD
SFT	SFT 8B + RL	TBD	TBD	TBD	TBD	TBD	TBD

G.3 Ablation Results

Reward variance is the mean within-group variance before GRPO normalization. Reachability collision is the fraction of sampled negative traces containing a state observed under an expanded execution sweep.

The reward variants correspond directly to the two levels of Equation 2. *Binary Inductiveness* assigns one iff the capped, normalized response is nonempty and every clause survives Houdini. *Whole-Rollout Strength* uses negative coverage only when every capped clause survives,

$$r_i^{\text{whole}} = \mathbf{1}[S_i = \text{dedup}(\bar{A}_i)]\frac{|R_i|}{|\mathcal{N}|} - w_o o(A_i).$$

Clause-Decomposed Strength removes this all-or-nothing indicator and scores the surviving S_i; *Compositional Credit* additionally adds $w_g \phi_i$; and *Regularized Compositional Credit* additionally subtracts $w_d d_{\text{dup}}(A_i)$, yielding the full reward. These are `binary`, `whole_coverage`, `base`, `base_shapley`, and `full`, respectively.

Table 11: Per-seed reward-ablation results on the 8B model. Seed columns and summary statistics report combine@10 verification rates.

Reward	Seed 1	Seed 2	Seed 3	Mean	Std.
Binary Inductiveness	TBD	TBD	TBD	TBD	TBD
Whole-Rollout Strength	TBD	TBD	TBD	TBD	TBD
Clause-Decomposed Strength	TBD	TBD	TBD	TBD	TBD
Compositional Credit	TBD	TBD	TBD	TBD	TBD
Regularized Compositional Credit	TBD	TBD	TBD	TBD	TBD

Table 12: Per-seed negative-trace sampler ablation on the 8B model. Seed columns and summary statistics report combine@10 verification rates.

Sampler	Seed 1	Seed 2	Seed 3	Mean	Std.
Random perturbation	TBD	TBD	TBD	TBD	TBD
Structured sampler	TBD	TBD	TBD	TBD	TBD

G.4 Cross-Model Main Results

Table 13: Complete main results by evaluation stratum. Each benchmark column reports the number verified by pass@1/combine@1.

Model	Linear / 316	NLA / 50	Loopy / 466	All / 832	Time / task	Total tokens / task
GPT-5-nano	193/203	7/7	252/294	452/504	66.60 s	6,928.30
GPT-5-mini	176/246	2/5	284/353	462/604	58.36 s	4,509.32
GPT-5	246/254	7/7	375/375	628/636	82.88 s	5,289.00
Claude Sonnet-4.6	TBD	TBD	TBD	467/528	59.88 s	1,193.55
DeepSeek-V4-Flash	183/196	2/4	243/254	428/454	166.98 s	11,084.44
Qwen3-14B	TBD	TBD	TBD	TBD	TBD	TBD
Qwen3-8B	TBD	TBD	TBD	TBD	TBD	TBD
Qwen3-4B	TBD	TBD	TBD	TBD	TBD	TBD
Llama (checkpoint TBD)	TBD	TBD	TBD	TBD	TBD	TBD

Claude timing and tokens use a fixed seed-0 stratified sample of 20 programs (8 linear, 1 nonlinear, and 11 Loopy); accuracy uses all 832. For the rows with retained paired artifacts, filter gain/regression counts are 123/71 (GPT-5-nano), 157/15 (GPT-5-mini), 23/15 (GPT-5), and 32/6 (DeepSeek-V4-Flash). These counts compare the same response before and after clause processing; they expose operational regressions hidden by the net rate.

As a reasoning-control diagnostic, GPT-5 with `reasoning_effort=none` obtains pass@1 of 251/832 (30.17%) and combine@1 of 477/832 (57.33%), using 1,028.12 mean tokens per task. The main table retains the provider-default run.

G.5 Comparison with Specialized Tools

Table 14: Complete target-hidden tool-comparison results. Each cell reports verified programs and rate under the common file-level denominator.

Method	Linear / 316	NLA / 50	Loopy / 466	Overall / 832	Tokens / task	Time / task
AutoSpec	208 (65.82%)	4 (8.00%)	302 (64.81%)	514 (61.78%)	25,154.54	54.77 s
SESpec	180 (56.96%)	3 (6.00%)	231 (49.57%)	414 (49.76%)	44,807.05	117.31 s
Clause2Inv	63 (19.94%)	1 (2.00%)	0 (0.00%)	64 (7.69%)	385.80	29.04 s
Daikon	73 (23.10%)	0 (0.00%)	101 (21.67%)	174 (20.91%)	0	4.89 s
Naive	154 (48.73%)	4 (8.00%)	221 (47.42%)	379 (45.55%)	4,493.37	28.87 s
LoopGym (pass@1)	193 (61.08%)	7 (14.00%)	252 (54.08%)	452 (54.33%)	6,928.30	53.52 s
LoopGym (combine@1)	203 (64.24%)	7 (14.00%)	294 (63.09%)	504 (60.58%)	6,928.30	66.60 s
LoopGym (combine@5)	250 (79.11%)	7 (14.00%)	365 (78.33%)	622 (74.76%)	33,404.56	284.81 s
LoopGym (combine@10)	255 (80.70%)	7 (14.00%)	376 (80.69%)	638 (76.68%)	68,953.35	536.81 s

Token totals are means per task, with three accounting qualifications. SESpec has exact token usage for 830 tasks. Clause2Inv token and time means use its 366 supported tasks, and its token count is estimated; its accuracy retains the 832-task denominator. The retained LoopGym pass@1 timing covers the 466 Loopy tasks. Pass@1 and combine@1 reuse the same first model response.

H Detailed Limitations

Program scope. Our implementation targets numerical programs with one scalar-integer loop. This scope follows the dominant setting in prior loop-invariant benchmarks, but excludes nested loops and invariants over arrays, heaps, or recursively defined predicates. Such invariants require richer representations and proof dependencies, while available training corpora contain too few examples to support the discriminative signal used here. Our state perturbations cannot yet construct reliably informative negatives for inductive predicates or interacting nested loops (Liu et al., 2024a).

Inference engine. In preliminary comparisons, locally evaluated checkpoints achieved lower verification rates under vLLM than under alternative backends despite nominally matched decoding settings. We nevertheless use vLLM for all local base and trained checkpoints because its throughput makes broad sampling practical and its consistent use avoids an additional train–test mismatch. API models retain service defaults. Local-model success rates may therefore be conservative rather than engine-independent ceilings.

Verifier and specification language. Our filtering and all final judgments use Frama-C/WP, and candidates use ACSL. Parser behavior, generated obligations, prover integration, and syntax affect which clauses survive. RCF is not intrinsically tied to this toolchain, but the quantitative conclusions may not transfer unchanged to other verifiers, specification languages, or source languages.

Finite candidate-pool approximation. Before Houdini, the combine implementation applies a lightweight AST-based filter to parse, normalize, and deduplicate candidate clauses. This filter is deliberately conservative and does not provide a completeness guarantee for the full ACSL expression language: a semantically useful clause that is not handled by its restricted parser can be omitted. In addition, combine@n caps the candidate union passed to verification at 300 clauses to keep WP cost bounded. When an extreme response pool exceeds this limit, later clauses may be truncated even if they add useful information. Both effects can make the reported combine@n underestimate an ideal unbounded implementation. They affect candidate recall, not the validity of reported successes, because every retained result must still pass the full Frama-C/WP final judge.

Feedback regime. We do not evaluate iterative optimization from verifier feedback. In pilot runs, models quickly reached valid but weak invariants: they passed inductiveness checks, but with Q hidden the verifier did not identify the missing behavioral strength. Refinement therefore yielded little marginal gain, which decreased further as larger initial rollout unions captured the easy corrections. Our results characterize target-independent training and finite-budget composition, not settings with rich counterexamples or target-directed repair.

I Why Thinking Is Disabled for Local Models

For API models, all thinking and reasoning settings are left at the service default; we do not send `reasoning_effort`, `thinking`, or an equivalent override. For all locally served checkpoints, including the base probes and models trained in our pipeline, we disable chain-of-thought (CoT) during synthetic data construction, training rollouts, and evaluation. Applying the same setting at all three stages avoids a train–test distribution shift. This choice has two practical motivations.

CoT substantially increases generation cost. Our inference procedure samples many rollouts for each program. Explicit CoT adds reasoning tokens and generation latency to every rollout, so its cost is multiplied by the sampling budget. The required output, however, is only a short, machine-parseable set of ACSL clauses. We therefore omit explicit CoT to keep both data generation and multi-rollout inference efficient.

Invariant clauses compose, but CoTs do not. A single rollout may contain both valid and invalid candidate invariants. The invariant output is explicitly decomposable into clauses: Houdini can remove a failing clause while retaining the useful clauses, and the retained clauses can then be combined across rollouts. A CoT does not have this property. Correct and incorrect reasoning may be interleaved in one trace, with no reliable clause-level correspondence that would let us

remove the reasoning associated with a rejected clause. Nor is there a principled way to merge the remaining fragments from different rollouts into one faithful CoT for the combined invariant set. Generating a new rationale after filtering would only produce a post-hoc explanation. Consequently, CoT supervision is not aligned with the clause-wise filter-and-combine target. For consistency, CoT remains disabled throughout the local-model pipeline.

J Training–Inference Mismatch in Houdini Ordering

Let A_i be the invariant set proposed by rollout i, and let $\mathsf{H}(X)$ denote syntax filtering followed by Houdini on candidate set X. SFT construction and inference first merge the candidates and then filter them:

$$S_{\text{infer}} = \mathsf{H}\left(\bigcup_{i=1}^{G} A_i\right). \tag{4}$$

This order allows a clause from one response to be preserved with the help of a companion clause proposed by another response.

RL requires a scalar reward for each rollout. The current implementation therefore runs Houdini independently and computes coverage, Shapley credit, and redundancy from

$$S_i^{\text{train}} = \mathsf{H}(A_i). \tag{5}$$

It also evaluates $\mathsf{H}(\bigcup_i A_i)$ as a group-level diagnostic, but this shared result does not determine each rollout's token-level advantage.

Why the two orders are not equivalent. Houdini is not distributive over union:

$$\mathsf{H}\left(\bigcup_i A_i\right) \neq \bigcup_i \mathsf{H}(A_i) \quad \text{in general.} \tag{6}$$

For example, a clause from one rollout may fail preservation in isolation but become inductive when conjoined with a supporting clause from another rollout. Thus, two clauses rejected by separate Houdini runs can belong to an inductive conjunction after union. Because the training reward filters each response first, it misses the positive credit associated with this cross-rollout support.

Why the practical effect is small. This is missed positive credit, not a dedicated penalty for non-inductiveness. The reward assigns coverage and Shapley credit to surviving clauses and penalizes duplicates and overflow. A unique clause that fails standalone Houdini incurs no additional penalty, although the absence of coverage can still give its rollout a lower group-relative advantage. At inference time, raw candidates from all responses are unioned before Houdini, so a complementary pair remains available and can be retained jointly. The mismatch therefore leaves inference-time cross-response support intact but may under-train clauses that depend on it. Pilot audits suggested that such rescues were rare, but we did not conduct a formal rescue-rate study; we therefore treat this as a qualitative observation, not an equivalence.

Why retain this approximation. Independent filtering gives a stable, parallelizable reward for every response. Assigning the same union-level score to all G responses is uninformative under group-relative normalization because the shared component cancels. Exact leave-one-out credit,

$$\Delta_i = V\left(\mathsf{H}\left(\bigcup_j A_j\right)\right) - V\left(\mathsf{H}\left(\bigcup_{j\neq i} A_j\right)\right),$$

requires at least $G+1$ union-level Houdini evaluations per prompt and can still miss higher-order interactions. Shapley-style credit captures such interactions more faithfully but requires many subset evaluations. This would be the Shapley value of the union-and-Houdini verifier game, unlike the closed-form Shapley allocation of negative-set coverage used by our reward.

Implications and future correction. The ordering difference makes standalone Houdini a slightly imperfect reward surrogate for combine@k, but it does not alter inference-time union-and-Houdini. Future work can report the rescue rate represented by the gap between the two sides of Equation 6 and test leave-one-out or randomized-subset credit when that rate is non-negligible. For the present experiments, we retain the cheaper approximation and leave that cost–fidelity tradeoff to future evaluation.